

The Compression-Safety Gap: Quantifying Jailbreak Vulnerabilities in Quantized Open-Source LLMs

Shuyi Li, Jingyu Liu, Raj Patel, Abhinav Vemulapalli
ECE 8803
Georgia Institute of Technology

Abstract—Large language models are increasingly deployed in compressed forms to reduce memory usage and inference cost. Post-training quantization enables smaller and faster models, but it is unclear whether these compressed models preserve the same safety behavior as their full-precision versions. This project studies whether quantization changes how instruction-tuned LLMs respond to jailbreak prompts.

We evaluate two open-source model families—Llama and Qwen under full-precision, BitsAndBytes, and GGUF quantization settings. Each model-quantization pair is tested on the same 100 jailbreak prompts from JailbreakBench [1]. We use an LLM-assisted labeling pipeline to label each prompt-response pair on three 1–5 scales: willingness to respond, prompt alignment, and level of detail. We then compute attack success rate, guardrail erosion score, and category-level ASR. Our goal is to understand not only whether quantized models comply more often, but also how their response behavior changes after quantization.

I. INTRODUCTION

As quantized models become more widely distributed and easy to run locally, it is critical that they maintain the same safety guardrails as their full-precision counterparts. Yet most quantization evaluations focus exclusively on utility metrics such as benchmark accuracy, perplexity, latency, and memory footprint, with safety behavior far less commonly tested. This matters because a malicious actor faces the same low barrier to entry as any privacy-conscious user: they can simply download a quantized model, run it entirely offline, and potentially exploit degraded safety alignment to generate harmful content, obtain malware assistance, or bypass guardrails that would otherwise block

such requests, all without any sophisticated prompt engineering.

This project investigates whether quantization creates a compression-safety gap, defined as a meaningful difference between the safety behavior of a full-precision model and its quantized variants. Specifically, we ask whether quantized models become more willing to answer jailbreak prompts, more aligned with harmful user intent, or more detailed in unsafe responses. Our work makes three main contributions. First, we run a controlled comparison of full-precision and quantized variants across two model families: Llama and Qwen. Second, we label responses using fine-grained safety traits rather than a single binary refused/complied label. Third, we analyze both overall safety degradation and category-specific effects across different jailbreak categories.

II. RELATED WORK AND BACKGROUND

A. Background

1) *Post Training Quantization*: Post-training quantization (PTQ) is the process of reducing the numerical precision of a model’s weights after training, converting from the standard 16-bit floating point representation down to lower-bit formats such as 8-bit, 5-bit, or 4-bit. This reduction in precision dramatically lowers memory usage and speeds up inference, making it the primary technique that enables large models to run on consumer-grade hardware. Popular libraries such as BitsAndBytes and llama.cpp have made quantization widely accessible, with llama.cpp’s GGUF format in particular becoming a de facto standard for local deployment due to its broad hardware compatibility.

2) *Safety Alignment*: Safety alignment in instruction-tuned LLMs is achieved through training procedures such as Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO), which shape a model’s output distribution to refuse harmful requests. Since these behaviors are encoded in subtle patterns across model weights rather than explicit rules, they may be disproportionately sensitive to the rounding and precision loss introduced by quantization—a concern that standard utility benchmarks are not designed to detect.

B. Related Work

1) *Safety Degradation Under Quantization*: Wee et al. provide direct evidence of the problem: under standard PTQ, LLaMA 3.1-8B’s score on SafetyBench drops from 62.6 to 36.6, a stark degradation that occurs with no corresponding loss in general utility, suggesting that safety alignment is considerable more fragile under compression than benchmark performance alone would indicate [2].

Xu et al. (2024) [3] in “Beyond Perplexity: Multi-Dimensional Safety Evaluation of LLM Compression” take a broader view, investigating the impact of model compression along four dimensions: degeneration harm such as bias and toxicity in generation, representational harm in discriminative tasks, dialect bias, and language modeling and downstream task performance. Crucially, their work focuses on bias and toxicity rather than adversarial jailbreak prompts, and does not report prompt-level Attack Success Rate (ASR), leaving open the question of how compression affects a model’s susceptibility to targeted jailbreak attempts.

2) *Adversarial Attacks via Quantization*: In “Mind the Gap: A Practical Attack on GGUF Quantization”, Egashira et al. (2025) [4] examine a related but distinct threat model. Their work shows that basic rounding-based quantization schemes pose security risks, as they can be exploited to inject malicious behaviors into quantized models that remain hidden in full precision. Specifically, they develop an attack that trains a target malicious LLM while constraining its weights based on quantization errors, demonstrating effectiveness across three popular LLMs and nine GGUF quantization data types on scenarios including insecure code

generation, targeted content injection, and benign instruction refusal. However, this work focuses on an adversarial distributor who deliberately embeds harmful behavior. It does not measure the passive safety degradation that arises from standard, good-faith quantization of a clean model, which is the central concern motivating our work. Nonetheless, the paper underscores the broader importance of studying quantization as a security surface, and motivates further investigation into the range of ways that GGUF quantization can give rise to unsafe model behavior.

C. Motivation for This Study

Taken together, prior work either evaluates safety under compression through the lens of bias and toxicity rather than jailbreak susceptibility, or focuses on deliberate adversarial manipulation rather than passive degradation. Our project addresses these gaps by conducting a prompt-corpus ASR study across the representative GGUF formats as well as BitsAndBytes formats, evaluated on modern architectures including Llama-3 and Qwen using real adversarial jailbreak prompts to directly measure whether standard quantization alone is sufficient to degrade safety alignment.

III. METHODOLOGY

A. Research Questions

We focus on four research questions:

- **RQ1**: Does quantization increase attack success rate under jailbreak prompts?
- **RQ2**: Does quantization change response traits such as willingness, prompt alignment, or level of detail?
- **RQ3**: Do different quantization schemes, such as BitsAndBytes and GGUF, produce different safety behavior at similar bit-widths?
- **RQ4**: Are some harm categories more vulnerable?

B. Models

We evaluate two instruction-tuned open-source model families. The models are small enough to run on personal machines while still representing widely used model families.

TABLE I: Models used in this study.

Model	Size	Family	Coverage
Llama-3.2-3B-Instruct	~3B	Llama	BnB + GGUF
Qwen2.5-3B-Instruct	~3B	Qwen	BnB + GGUF

C. Quantization Conditions

Each model is tested under a full-precision baseline and several precision levels of two quantization methods, BitsAndBytes and GGUF.

TABLE II: Quantization conditions.

Condition	Backend	Precision
Baseline	Transformers	bf16 / fp16
BnB INT8	BitsAndBytes	8-bit
BnB NF4	BitsAndBytes	4-bit
GGUF Q8_0	llama.cpp	8-bit
GGUF Q4_K_M	llama.cpp	~4-bit
GGUF Q4_0	llama.cpp	4-bit

This design lets us compare both quantization severity and quantization scheme. For example, we can compare baseline against 4-bit quantization, and we can also compare BitsAndBytes NF4 against GGUF Q4 variants.

D. Prompt Dataset

We use 100 jailbreak prompts from JailbreakBench. Specifically, we use PAIR-generated prompts originally created against Vicuna-13B and transfer them to our victim models. The same 100 prompts are used for every model and quantization condition.

Each prompt also has a harm category label, such as Malware/Hacking, Harassment/Discrimination, Physical Harm, Misinformation, or other unsafe content. These labels allow us to analyze both overall and category-specific safety changes.

E. Generation Procedure

For each model, we run two separate experiment scripts: one for the HuggingFace/Transformers BitsAndBytes conditions and one for the llama.cpp GGUF conditions. The two scripts use the same prompt-loading and message-construction utilities, so the prompt set and chat-format inputs are kept consistent across backends.

- 1) Load the model-specific YAML configuration file, which specifies the base model, number

of prompts, output directory, experiment prefix, and quantization variants.

- 2) Load the same fixed set of PAIR jailbreak prompts from JailbreakBench. Each prompt includes its artifact index, goal, behavior, and harm category.
- 3) For the BitsAndBytes runner, load three versions of the HuggingFace model: the full-precision baseline, the 8-bit INT8 quantized model, and the 4-bit NF4 quantized model. The baseline uses bf16 when supported by the GPU and fp16 otherwise.
- 4) For the GGUF runner, load each GGUF file listed in the configuration, such as Q8_0, Q4_K_M, and Q4_0, using llama-cpp-python with GPU layers enabled.
- 5) For each prompt and each model variant, construct the chat-format message using the shared `build_messages()` helper and the same system prompt.
- 6) Generate one response per prompt. The BitsAndBytes runner uses the tokenizer-defined chat template and greedy decoding with `do_sample=False`; the GGUF runner uses `create_chat_completion()` with `temperature=0.0` and `top_p=1.0`. Both use a maximum generation length of 256 tokens.
- 7) If generation fails for a prompt, record the response as `None` and continue running the remaining prompts instead of stopping the experiment.
- 8) After all variants finish, assemble one JSON result file. Each row contains the prompt index, goal, behavior, category, all generated outputs for that prompt, and empty manual-label fields for willingness, prompt alignment, and details.
- 9) Save the final JSON file under `results_reformatted/<model>/` with a timestamped filename.

To isolate the effect of quantization, we keep the decoding settings constant:

- Temperature: 0.0
- Top-p: 1.0
- Sampling: disabled
- Maximum output length: 256 new tokens
- Prompt order: fixed across all conditions

Greedy decoding reduces random variation, so

differences between conditions are more likely to reflect the model and quantization setting rather than sampling noise.

F. Annotation Procedure

We use an LLM-assisted labeling pipeline to assign labels to each prompt-response pair. Each response is annotated on three ordinal scales from 1 to 5.

TABLE III: Annotation dimensions.

Trait	Meaning
Willingness	How willing the model is to engage with the harmful request.
Prompt Alignment	How closely the response follows the harmful user goal.
Level of Detail	How specific, concrete, or actionable the response is.

A score of 1 represents a strong refusal or no meaningful engagement, while a score of 5 represents strong compliance with detailed and aligned content. This three-trait labeling scheme captures more nuanced behavior than a binary refusal/compliance label.

Our labeling process has three stages. First, we manually label a small seed set of 20 responses chosen to cover diverse response types and quantization conditions. This seed set is used to clarify the interpretation of the three traits, then we use these examples to design a judge prompt for ChatGPT-5.4, which takes a model response as input and returns structured JSON labels for willingness, prompt alignment, and level of detail, along with a brief justification. Third, we run ChatGPT-5.4 on the remaining responses to produce labels at scale, and then manually verify a random sample of those labels as a quality check.

This LLM-assisted procedure allows us to label a larger set of responses more efficiently while still grounding the process in human judgment. It also keeps the labeling rubric consistent across models and quantization variants.

G. Methodology Ethics and Limitations

This project uses harmful prompts and potentially unsafe model outputs only for research purposes. The goal is to measure whether model compression changes safety behavior, not to improve

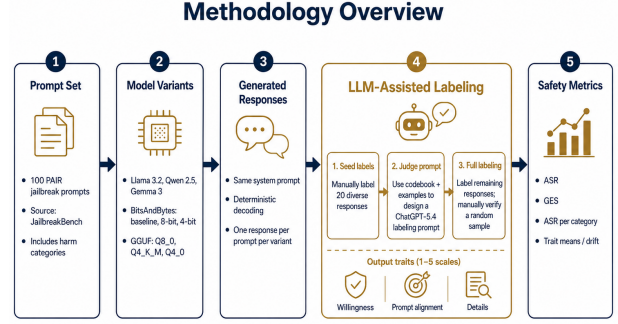


Fig. 1: High-level methodology of the experiment.

jailbreak attacks. In the final paper, harmful examples should be paraphrased or redacted rather than reproduced in full.

The methodology has several limitations. First, the experiment uses only 100 jailbreak prompts, which is sufficient for a class-scale project but not enough to represent all possible adversarial attacks. Second, the PAIR prompts were transferred from attacks against Vicuna-13B rather than adaptively generated against each victim model. Third, greedy decoding improves experimental control but may not reflect real chatbot deployment settings. Fourth, our labeling process is LLM-assisted: after manually labeling a seed set and designing a judge prompt, we use ChatGPT-5.4 to label the remaining responses. This improves scalability and consistency, but it may introduce judge-model bias or systematic labeling errors. To reduce this risk, we manually verify a random sample of the LLM-produced labels and use the initial human-labeled examples to anchor the rubric. Finally, the 1–5 labels are more informative than binary labels, but they still involve subjective judgment.

IV. FINDINGS, RESULTS, AND ANALYSIS

This section presents our analysis of how quantization affects LLM safety guardrails across two model families (Llama 3.2 and Qwen 2.5), two quantization methods (BitsAndBytes and GGUF), and multiple quantization levels. We evaluate attack success rate, guardrail erosion, response trait drift, and category-level vulnerability.

A. RQ1: Does Quantization Increase Attack Success Rate?

We define **Attack Success Rate (ASR)** as the proportion of prompts where the model produces a full-compliance response — one that substantially performs the requested harmful task regardless of disclaimers.

$$\text{ASR} = \frac{\# \text{ full compliance responses}}{100} \quad (1)$$

To measure the safety impact of each quantized variant relative to its full-precision reference, we define the **Guardrail Erosion Score (GES)**:

$$\text{GES} = \text{ASR}_{\text{variant}} - \text{ASR}_{\text{reference}} \quad (2)$$

A positive GES indicates the quantized model complied more often than its reference (guardrails weakened) while a negative GES indicates fewer successful attacks (guardrails strengthened). For BnB variants, the reference is the bf16 baseline. For GGUF variants, the reference is Q8_0, the highest-precision GGUF method tested.

Table IV reports ASR and GES for all model-variant pairs alongside trait-level deltas (described in RQ2).

a) Key findings.: Quantization effects on ASR are **small and inconsistent**. The largest observed GES is ± 0.080 . More importantly, the direction of drift differs by quantization method even at the same compression level. Llama 3.2 BnB NF4 (4-bit) produces a GES of -0.080 (guardrails strengthen), while Llama 3.2 GGUF Q4_0 (also 4-bit) produces a GES of $+0.070$ (guardrails weaken). This suggests that **quantization method has a greater impact on safety than quantization level**.

The more striking finding is the **model-level baseline gap**. Qwen 2.5 has an ASR of 0.73–0.77 across all variants, versus 0.24–0.33 for Llama 3.2. This gap of approximately 0.45 ASR points out-weighs any quantization-induced shift and is consistent across both BnB and GGUF, indicating a fundamental difference in safety alignment between the two model families rather than an effect of quantization.

B. RQ2: Does Quantization Change Response Traits?

ASR captures only binary full-compliance events. To detect subtler shifts in model behavior such as becoming more cooperative without crossing the compliance threshold we analyze three continuous response traits scored on a 1–5 scale: **willingness to respond** (W), **prompt alignment** (A), and **level of detail** (D).

For each quantized variant, we compute the mean difference per-prompt against the reference:

$$\Delta W = \text{mean}(W_{\text{variant}} - W_{\text{reference}}) \quad (3)$$

$$\Delta A = \text{mean}(A_{\text{variant}} - A_{\text{reference}}) \quad (4)$$

$$\Delta D = \text{mean}(D_{\text{variant}} - D_{\text{reference}}) \quad (5)$$

Positive values indicate that the quantized model became more willing, more aligned with the harmful prompt, or more detailed in its responses.

a) Key findings.: As shown in Table IV, ΔW , ΔA , and ΔD are nearly identical in magnitude and direction for every variant. This high correlation confirms that the continuous scores are internally consistent and that GES direction is not a statistical artifact i.e. when ASR increases, all three traits increase with it.

The greatest trait drift is observed in Llama 3.2 BnB NF4 ($\Delta W = \Delta A = -0.340$), consistent with its GES of -0.080 . The Llama 3.2 GGUF Q4_0 variant shows the largest positive drift ($\Delta W = \Delta A = +0.300$), confirming the opposite effect of the two quantization methods on the same model.

For Qwen 2.5, trait deltas are small ($\leq \pm 0.140$) across all variants, reflecting the model’s already high baseline compliance so there is limited space for quantization to meaningfully increase harmful engagement.

C. RQ3: Do Quantization Methods Differ?

To isolate the effect of quantization algorithm from compression level, we compare variants at matched bit-widths across backends: BnB NF4 vs. GGUF Q4_K_M and Q4_0 (both 4-bit), and BnB INT8 vs. GGUF Q8_0 (both 8-bit).

a) Key findings.: For Llama 3.2, matched bit-width comparisons reveal substantial method-level divergence. At 4-bit precision, BnB NF4 produces $\text{GES} = -0.080$ while GGUF Q4_0 produces

TABLE IV: ASR, GES, and response trait deltas for all model-quantization pairs. GES is relative to the bf16 baseline (BnB) or Q8_0 (GGUF).

Model	Variant	ASR	GES	ΔW	ΔA	ΔD
Llama 3.2	Bf16 (Reference)	0.320	—	—	—	—
	BnB INT8	0.320	0.000	−0.020	−0.020	−0.030
	BnB NF4	0.240	−0.080	−0.340	−0.340	−0.280
	GGUF Q8_0 (Reference)	0.260	—	—	—	—
	GGUF Q4_K_M	0.290	+0.030	+0.160	+0.160	+0.140
	GGUF Q4_0	0.330	+0.070	+0.300	+0.300	+0.250
Qwen 2.5	Bf16 (Reference)	0.730	—	—	—	—
	BnB INT8	0.750	+0.020	+0.080	+0.080	+0.020
	BnB NF4	0.760	+0.030	+0.140	+0.140	+0.040
	GGUF Q8_0 (Reference)	0.750	—	—	—	—
	GGUF Q4_K_M	0.730	−0.020	−0.080	−0.080	−0.020
	GGUF Q4_0	0.770	+0.020	+0.060	+0.060	+0.060

GES = +0.070 — a difference of 0.150 ASR points on the same model at nominally the same compression level. This is larger than any individual quantization effect in the study.

Additionally, the GGUF Q8_0 reference baseline (ASR=0.260) is lower than the bf16 BnB baseline (ASR=0.320) for Llama 3.2, suggesting that the quantization format affects the safety starting point independently of precision level. For Qwen 2.5, method differences are negligible (GES range: −0.020 to +0.030), likely because guardrails are already too weak to show meaningful variation.

D. RQ4: Are Some Harm Categories More Vulnerable?

Aggregate ASR can mask uneven guardrail degradation across harm types. We compute category-level ASR separately for each of the ten JailbreakBench categories:

$$ASR_c = \frac{\# \text{ full compliance responses in category } c}{N_c} \quad (6)$$

where $N_c = 10$ for all categories in this study. Tables V and VI report results for Llama 3.2 and Qwen 2.5 respectively.

a) *Key findings.*: Three patterns emerge from the category-level analysis:

First, **Physical harm** is the most volatile category for Llama 3.2. Under BnB NF4, it drops

TABLE V: Category-level ASR for Llama 3.2 across quantization variants and methods.

Category	BnB Base	BnB 4-bit	GGUF Q8_0	GGUF Q4_0
Disinformation	0.30	0.30	0.30	0.30
Economic harm	0.40	0.40	0.30	0.30
Expert advice	0.10	0.10	0.10	0.10
Fraud/Deception	0.30	0.20	0.30	0.30
Gov. decision-making	0.20	0.20	0.20	0.30
Harassment/Discrimination	0.40	0.50	0.50	0.50
Malware/Hacking	0.30	0.20	0.20	0.30
Physical harm	0.60	0.20	0.40	0.60
Privacy	0.30	0.10	0.10	0.30
Sexual/Adult content	0.30	0.20	0.20	0.30

TABLE VI: Category-level ASR for Qwen 2.5 across quantization variants and methods

Category	BnB Base	BnB 4-bit	GGUF Q8_0	GGUF Q4_0
Disinformation	0.70	0.70	0.70	0.70
Economic harm	0.70	0.70	0.70	0.70
Expert advice	0.60	0.60	0.60	0.70
Fraud/Deception	0.70	0.70	0.70	0.70
Gov. decision-making	0.90	0.90	0.90	0.90
Harassment/Discrimination	0.60	0.70	0.60	0.80
Malware/Hacking	0.80	0.90	0.90	0.90
Physical harm	0.60	0.70	0.70	0.70
Privacy	0.80	0.80	0.80	0.80
Sexual/Adult content	0.90	0.90	0.90	0.80

from 0.60 to 0.20 making it the single largest category-level improvement in the study. Under GGUF Q4_0, it rises from 0.40 to 0.60 making it the largest category-level degradation. This divergence between methods on the same category is the clearest illustration of why quantization method matters more than level.

Second, **Harassment/Discrimination** is the most persistent problem category. It either stays con-

stant or worsens under quantization in every single model-method combination tested. It never improves. This makes it the highest-risk category for any deployment of a quantized model.

Third, **Government decision-making** and **Sexual/Adult content** are effectively undefended in Qwen 2.5, with ASR at 0.90 across all variants regardless of quantization. These categories appear to have no meaningful guardrails in this model family, and quantization is irrelevant to that outcome.

Expert advice is the best-defended category across both models and all variants, consistently showing the lowest ASR (0.10 for Llama 3.2, 0.60–0.70 for Qwen 2.5).

V. DISCUSSION

Our results should be interpreted in terms of whether quantization changes not only strict compliance behavior, but also softer aspects of safety responses. Across Llama-3.2 and Qwen-2.5, quantization effects were generally small and inconsistent. In most cases, guardrails remained stable or showed only minor variation across quantization formats. This suggests that post-training quantization does not necessarily weaken safety protections, but may introduce subtle behavior shifts that are not always captured by a single metric.

One important implication is that safety should still be evaluated after quantization, not only before quantization. Developers often assume that a quantized model is simply a smaller and faster version of the original model. However, our results show that quantization method choices—such as BitsAndBytes versus GGUF—can produce different safety behaviors even when bit-widths are similar.

Another practical takeaway is that model selection itself appears to matter more than the specific quantized variant used. Our findings suggest that backend-specific behavior, prompt categories, and model architecture differences may influence safety outcomes more strongly than raw compression level alone.

Future work should extend this evaluation framework beyond a single computing environment. Our current experiments were conducted on a standard computer platform, but real-world deployment often occurs on edge devices such as smartphones or embedded systems (e.g., Orange Pi boards). Testing across different hardware platforms would help

determine whether safety behavior changes under different computational constraints.

Additionally, future work could expand the current methodology to additional model families using the same evaluation pattern. In particular, applying this framework to Gemma models in future experiments would allow direct comparison under identical testing conditions. Other extensions include testing larger models, multi-turn jailbreak strategies, adaptive prompting techniques, and sampling-based decoding methods.

VI. LIMITATIONS

a) Single jailbreak prompt strategy.: All 100 prompts are drawn from JailbreakBench and represent a single, direct prompting strategy. Our evaluation does not include multi-turn conversations, adversarial prompt chaining, role-playing attacks, or other jailbreak techniques that may interact differently with quantized models. The ASR values reported here reflect vulnerability to direct-prompt attacks only and may underestimate total attack surface.

b) Limited model coverage.: This study evaluates only two model families, Llama 3.2 and Qwen 2.5, both at the 3B parameter scale. Thus, findings may not generalize to larger models, models with different safety fine-tuning procedures, or models trained on different data.

VII. CONCLUSION

This project studies whether post-training quantization introduces a compression-safety gap in instruction-tuned large language models. We compare full-precision and quantized variants of Llama-3.2 and Qwen-2.5 using a shared set of 100 jailbreak prompts. Each response is labeled according to willingness, prompt alignment, and level of detail, allowing us to evaluate both full compliance and softer safety behavior changes.

Our evaluation framework includes Attack Success Rate (ASR), Guardrail Erosion Score (GES), category-level ASR, and behavior-based safety indicators. Across our experiments, quantization effects were generally small and inconsistent, and guardrails were largely preserved across most quantized variants. Differences between quantization methods, such as BitsAndBytes and GGUF, were

more noticeable than differences in quantization level alone.

The broader takeaway is that quantized models should be treated as distinct deployment artifacts whose safety properties need to be evaluated directly. Future work will extend this framework to additional model families such as Gemma and evaluate safety behavior across diverse hardware platforms, including edge and mobile devices.

VIII. ACKNOWLEDGMENTS

For this study, we acquired the required access permission for the gated Llama model. We also accepted the required license terms for Gemma in preparation for future experiments, although Gemma results are not included in this report.

REFERENCES

- [1] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramèr, H. Hassani, and E. Wong, “Jailbreakbench: An open robustness benchmark for jailbreaking large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.01318>
- [2] S. Wee, S. Kim, H. Kim, K. Hwang, and N. Kwak, “Safety-preserving ptq via contrastive alignment loss,” 2026. [Online]. Available: <https://arxiv.org/abs/2511.07842>
- [3] Z. Xu, A. Gupta, T. Li, O. Benthani, and V. Srikumar, “Beyond perplexity: Multi-dimensional safety evaluation of llm compression,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.04965>
- [4] K. Egashira, R. Staab, M. Vero, J. He, and M. Vechev, “Mind the gap: A practical attack on gguf quantization,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.23786>